AburráSegura

Guía del proyecto explicada de forma sencilla

Qué es cada parte, para qué sirve y cómo trabajan juntas — sin lenguaje técnico

Proyecto Pedagógico Integrador (PPI) · Análisis de Algoritmos
Tecnológico de Antioquia

1. ¿Qué es AburráSegura?

AburráSegura es una página web que ayuda a moverse por Medellín de forma **más segura**. Usa información real de la Alcaldía (MEData) sobre **251.150 accidentes de tránsito** ocurridos entre 2014 y 2020.

Con esa información, la aplicación puede:

• Mostrar en un mapa **dónde ocurren más accidentes** (zonas "calientes").
• Calcular dos caminos entre un punto A y un punto B: el **más corto** y el **más seguro** (el que pasa por calles con menos accidentes).
• Mostrar **estadísticas**: a qué horas, qué días y en qué barrios hay más accidentes.
• Funcionar como un **laboratorio** para comparar qué tan rápidos son distintos métodos de cálculo (esto es la parte académica del curso).

> **Idea clave:** piensa en la aplicación como un "Waze de la seguridad vial": en vez de buscar solo el camino más rápido, también te dice cuál camino tiene menos historial de accidentes.

1.1 Las tres grandes piezas

Para entender el proyecto, sirve imaginar un **restaurante**:

Archivo	¿Qué hace? (en palabras simples)
La sala (frontend)	Es lo que ve el usuario: el mapa, los botones, las gráficas. Está en la carpeta **frontend/**. Como el mesero, recibe lo que pides y te muestra el resultado bonito.
La cocina (backend)	Es donde se hace el trabajo pesado: buscar datos, calcular rutas, contar accidentes. Está en la carpeta **app/**. El usuario nunca la ve, pero ahí se "cocina" todo.
La despensa (datos)	Son los archivos con la información de los accidentes y el mapa de calles. Está en la carpeta **data/**. Es la materia prima de la cocina.

La sala le pide cosas a la cocina por medio de "pedidos" (en informática se llaman *peticiones a la API*). La cocina responde con los datos ya preparados.

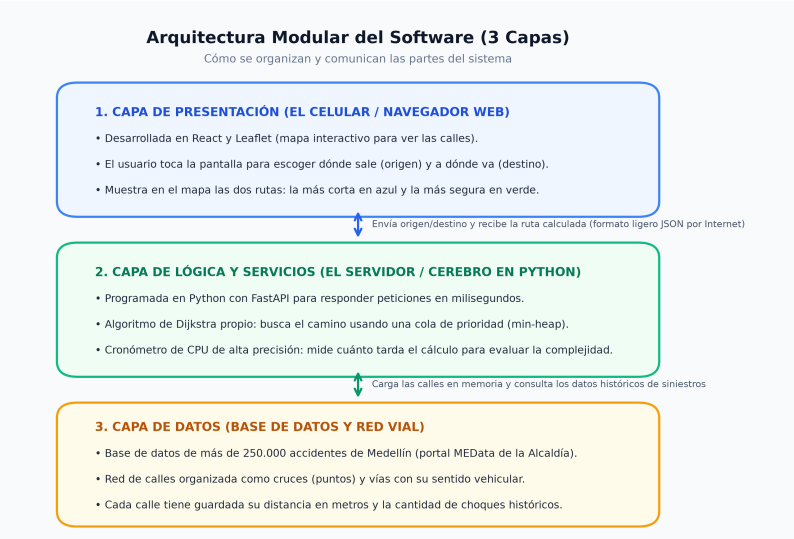


Figura 1. Las capas del sistema (incluida en la carpeta diagramas/).

2. El mapa de carpetas

Así está organizado el proyecto. Cada carpeta tiene una función clara:

Archivo	¿Qué hace? (en palabras simples)
app/	La cocina: el servidor que hace los cálculos (escrito en Python).
app/algorithms/	Las "recetas" matemáticas: los métodos para ordenar, buscar, agrupar y encontrar caminos.
app/api/	La "ventanilla de pedidos": define qué cosas se le pueden pedir al servidor.
app/services/	Los "cocineros": toman el pedido, consultan la despensa y usan las recetas.
app/models/	Los "formularios": definen cómo debe venir cada pedido y cada respuesta.
frontend/	La sala: la página web que ve el usuario (escrita en JavaScript con React).
data/	La despensa: accidentes, mapa de calles y datos de prueba.
scripts/	Herramientas sueltas: descargar datos, limpiarlos, generar informes, publicar la app.
tests/	Pruebas automáticas que revisan que todo funcione bien.
diagramas/	Imágenes que explican el sistema.
vault/	Notas de documentación académica (para el programa Obsidian).
.agents/	Instrucciones para asistentes de inteligencia artificial que ayudaron en el desarrollo. No afecta la app.
.vscode/	Configuración del editor Visual Studio Code (por ejemplo, el botón para encender todo).
.venv/ y node_modules/	Programas de terceros descargados automáticamente. No se tocan.

3. La despensa: los datos (carpeta data/)

Toda la aplicación depende de la información de accidentes. Esta llega "cruda" y hay que limpiarla antes de usarla, igual que se lavan y pican los ingredientes antes de cocinar.

Archivo	¿Qué hace? (en palabras simples)
data/raw/incidentes_viales.csv	El archivo original descargado de MEData (datos abiertos de Medellín). Es como una gran hoja de Excel con un accidente por fila.
data/raw/download_metadata.txt	Una nota que dice de dónde y cuándo se descargó el archivo, para poder comprobar que es el original.
data/processed/medellin_incidents_clean.csv	La versión **limpia** de los accidentes: sin errores, con coordenadas válidas y textos corregidos.
data/processed/medellin_incidents.db	Una **base de datos** (como un archivador muy ordenado) creada a partir del archivo limpio. Permite buscar datos muy rápido. Si falta o está dañada, la aplicación la vuelve a crear sola al arrancar.
data/processed/medellin_graph.json	El **mapa de calles** de Medellín convertido en una red de puntos (esquinas) y líneas (tramos de calle). Cada tramo tiene anotado cuántos accidentes ha tenido.
data/benchmarks/bench_*.csv	Muestras de distintos tamaños (1 mil, 10 mil, 50 mil, 100 mil, 500 mil y 1 millón de registros) usadas para medir velocidades.
data/benchmarks/benchmark_results_fase4.json	Resultados guardados de esas mediciones de velocidad.

Detalle curioso: el archivo original tenía errores de escritura. Por ejemplo, "SOLO DAÑOS" aparecía escrito de formas raras ("SOLO DAÃ±OS") y "Belén" aparecía como "BelÃ©n". El proyecto corrige eso automáticamente para que las cuentas salgan bien.

4. Los algoritmos, explicados fácil (carpeta app/algorithms/)

Un **algoritmo** es una receta: una lista de pasos que se siguen en orden para resolver un problema. En este proyecto todos se programaron "a mano", desde cero, porque el objetivo del curso es entender cómo funcionan por dentro. Cada uno resuelve una tarea distinta.

4.1 Dijkstra: encontrar el mejor camino

Para qué se usa: calcular la ruta del punto A al punto B. Está en **graph_routing.py**.

Cómo funciona: el programa ve la ciudad como una telaraña. Cada esquina es un nudo y cada calle es un hilo entre dos nudos. Cada calle tiene un "precio" (qué tan larga es y qué tan peligrosa).

Es como echar una **gota de tinta** en el punto de origen:

1. La tinta avanza primero por las calles más baratas.
2. Cada vez que llega a una esquina, anota cuánto le costó llegar ahí.
3. Si después encuentra un camino más barato a esa misma esquina, corrige la nota.
4. Sigue así hasta que la tinta llega al destino.

> Como siempre avanza por lo más barato, cuando llega al destino ya sabe que encontró el camino de menor costo.

4.2 A* (A estrella): Dijkstra con brújula

Para qué se usa: lo mismo que Dijkstra, pero más rápido. También está en **graph_routing.py**.

Cómo funciona: Dijkstra avanza en todas las direcciones, incluso hacia donde no está el destino. A* tiene una **brújula**: también se pregunta "¿esta esquina me acerca a donde voy?".

Si vas del centro a Envigado, A* casi no pierde tiempo explorando calles de Bello. Llega al mismo resultado revisando muchas menos calles.

4.3 Ruta corta vs. ruta segura: cómo se combinan

Cada calle tiene dos números: la **distancia** y el **riesgo** (cuántos accidentes ha tenido). El usuario elige cuánto le importa cada uno, por ejemplo 30% distancia y 70% seguridad, y el programa mezcla los dos números con esas proporciones.

• **Ruta más corta:** solo cuenta la distancia.
• **Ruta más segura:** le da más peso al riesgo, así que puede dar un pequeño rodeo para evitar calles peligrosas.

> Es como elegir entre la autopista rápida pero con muchos choques o una calle un poco más larga pero tranquila.

4.4 KD-Tree: encontrar lo más cercano muy rápido

Para qué se usa: cuando haces clic en el mapa, encontrar la esquina más cercana entre miles. Está en **spatial_kdtree.py**.

Cómo funciona: revisar las esquinas una por una sería lento. El KD-Tree juega a **"adivina quién"**:

1. ¿El punto está al norte o al sur de la mitad? Sur: se descarta medio mapa.
2. ¿Está al oriente o al occidente? Oriente: se descarta otra mitad.
3. Se repite norte/sur, oriente/occidente…

Con unas **15 o 20 preguntas** encuentra el punto entre cientos de miles. Es como buscar una palabra en el diccionario: no lees todas las páginas, abres por la mitad y vas descartando.

Distancia Haversine: junto con el KD-Tree se usa esta fórmula para medir la distancia real en metros entre dos puntos. Tiene en cuenta que la Tierra es redonda y no una hoja plana.

4.5 DBSCAN: encontrar los puntos negros

Para qué se usa: descubrir los lugares de Medellín donde se amontonan los accidentes. Está en **clustering_dbscan.py**.

Cómo funciona: imagina que cada accidente es una persona parada en un parque:

1. Se toma un accidente y se pregunta: "¿hay al menos 10 accidentes más a menos de 150 metros?"
2. Si **sí**, ahí empieza un **grupo** (un punto negro).
3. Se revisan los vecinos de ese grupo, y los vecinos de los vecinos, y el grupo crece mientras siga habiendo accidentes amontonados.
4. Si un accidente está **solo**, sin muchos alrededor, se considera "ruido" y se ignora.

Al final quedan los "racimos" de accidentes, que son las zonas peligrosas. Para cada racimo se calcula su punto central y qué tan graves fueron los accidentes.

4.6 Los métodos para ordenar

Para qué se usan: ordenar datos de menor a mayor y comparar cuál método es más rápido (es la parte académica del laboratorio). Están en **sorting.py**. Todos dan el mismo resultado; lo que cambia es la velocidad.

Archivo	¿Qué hace? (en palabras simples)
Merge Sort (mezcla)	Partes un mazo de cartas en mitades, y esas en mitades, hasta tener cartas sueltas. Luego las vas juntando de dos en dos, siempre ordenadas. Juntar dos montones ya ordenados es muy fácil.
Quick Sort (rápido)	Escoges una carta "jefe" (el pivote). Las menores van a la izquierda y las mayores a la derecha. Repites lo mismo en cada lado.
Heap Sort (montículo)	Como un **torneo**: el número más grande siempre gana y sube a la cima. Lo sacas, se juega otra ronda, sale el siguiente más grande, y así.

Archivo	¿Qué hace? (en palabras simples)
Counting Sort (conteo)	No compara nada, **cuenta**: "hay tres 1, cinco 2, dos 3…" y luego escribe la lista en orden. Es rapidísimo cuando hay pocos valores distintos y muy repetidos.
Timsort	El método que trae Python por defecto. Aprovecha los pedazos que ya vienen ordenados. Se usa como punto de comparación.

El laboratorio también prueba cada método con datos **desordenados, casi ordenados, al revés y con muchos repetidos**, porque algunos métodos se comportan mejor o peor según cómo vengan los datos.

4.7 La escalera de velocidades (de O(1) a O(n²))

Esta parte, en **complexity_benchmarks.py**, demuestra que **no todos los métodos aguantan igual cuando los datos crecen**. Con estas letras se describe qué tan rápido se vuelve lento un método:

Nombre	Ejemplo cotidiano	Si duplicas los datos…
O(1), instantáneo	Abrir un libro en la página que ya conoces. En el proyecto: buscar un accidente por su número.	…tarda **lo mismo**
O(log n), muy rápido	Buscar en un diccionario abriendo por la mitad.	…tarda **solo un pasito más**
O(n), normal	Revisar una fila de personas, una por una.	…tarda **el doble**
O(n log n), algo más lento	Lo que tardan los buenos métodos de ordenar (Merge, Quick, Heap).	…tarda **un poco más del doble**
O(n²), muy lento	Que cada persona de un salón salude a todas las demás.	…tarda **cuatro veces más**

> Con 250.000 accidentes, la diferencia es enorme: algo O(1) responde al instante, mientras que algo O(n²) podría tardar horas.

4.8 En resumen

- **Dijkstra / A*:** encuentran el camino (la tinta y la brújula).
- **KD-Tree:** encuentra lo más cercano rápido ("adivina quién").
- **DBSCAN:** encuentra dónde se amontonan los accidentes.
- **Métodos de ordenar:** ponen los datos en orden; el laboratorio compara su velocidad.
- **La escalera O(1)…O(n²):** muestra por qué elegir bien el método importa tanto.

5. La cocina: el servidor (carpeta app/)

El servidor es un programa que está siempre encendido esperando pedidos. Cuando la página web le pide algo, él lo prepara y responde. Está hecho con una herramienta llamada **FastAPI**.

5.1 Archivos principales

Archivo	¿Qué hace? (en palabras simples)
app/main.py	El **interruptor principal** del servidor. Lo enciende, le pone nombre y le dice quién puede hacerle pedidos (por seguridad). También tiene un "¿estás vivo?" (/health) que la página usa para saber si el servidor responde.
app/api/api_router.py	El **recepcionista**: junta todas las ventanillas de pedidos en un solo lugar.
app/models/schemas.py	Los **formularios** oficiales. Revisan que cada pedido venga completo y con valores razonables (por ejemplo, que no pidan ordenar 10 millones de datos y tumben el servidor).

5.2 Las ventanillas de pedidos (app/api/v1/)

Archivo	¿Qué hace? (en palabras simples)
incidents.py	Pedidos sobre **accidentes**: listarlos, resúmenes, estadísticas con filtros (año, gravedad, comuna), puntos críticos y mapa de calor.
routing.py	Pedidos sobre **rutas**: buscar una dirección escrita ("Universidad de Antioquia"), convertir un clic del mapa en dirección, y calcular la ruta corta vs. la ruta segura.
clusters.py	Pedido para encontrar **puntos negros** con DBSCAN.
benchmarks.py	Pedidos del **laboratorio**: medir velocidad de ordenamientos, del KD-Tree y de la jerarquía de complejidades.

5.3 Los cocineros (app/services/)

Archivo	¿Qué hace? (en palabras simples)
data_service.py	El cocinero principal. Abre la base de datos, carga el mapa de calles, busca lugares (tiene una lista propia de sitios conocidos de Medellín y entiende abreviaturas como "cra" = carrera o "cll" = calle), calcula rutas, arma el mapa de calor y las estadísticas.
benchmark_service.py	El encargado del laboratorio: toma las muestras de datos, ejecuta los métodos y mide cuánto tardan.

Archivo	¿Qué hace? (en palabras simples)
db_builder.py	El que **arma la base de datos** desde el archivo limpio y corrige los textos mal escritos. Se ejecuta solo si la base no existe.

6. La sala: la página web (carpeta frontend/)

Es lo que el usuario ve en el navegador. Está hecha con **React** (una herramienta para construir páginas por "piezas" o componentes) y **Leaflet** (para dibujar mapas).

6.1 Archivos base

Archivo	¿Qué hace? (en palabras simples)
index.html	La hoja en blanco donde se dibuja toda la aplicación.
src/main.jsx	El que "pega" la aplicación en esa hoja en blanco. Es el punto de arranque.
src/App.jsx	El **organizador de la página**. Decide qué pestaña mostrar (Mapa, Laboratorio, Analítica o Ficha Técnica), recuerda la pestaña en la dirección del navegador (#map, #lab...) y pone el pie de página.
src/api.js	Guarda la **dirección del servidor**, para que la página sepa a dónde enviar los pedidos.
src/index.css	Los **estilos**: colores, letras, tamaños, espacios. Es la "decoración" de la sala.
vite.config.js	Configuración de la herramienta que arma la página mientras se desarrolla (Vite). Conecta la página con el servidor local.
package.json	La lista de herramientas externas que necesita la página.

6.2 Las piezas de la pantalla (src/components/)

Archivo	¿Qué hace? (en palabras simples)
Navbar.jsx	La **barra de arriba**: el logo, los botones de las pestañas y un indicador que dice si el servidor está conectado, desconectado o "despertando" (el servidor gratuito se duerme si nadie lo usa).
MapView.jsx	La pestaña **Rutas & Mapa Seguro**. Es la más grande. Muestra el mapa, el buscador de origen (A) y destino (B), el mapa de calor, los puntos críticos con su nivel de riesgo (Bajo, Moderado, Alto, Crítico, Extremo) y dibuja las dos rutas para compararlas.
AlgorithmLab.jsx	La pestaña **Laboratorio Algorítmico**. Permite escoger un método y una cantidad de datos, ejecutarlo y ver cuánto tardó.
ComplexityCharts.jsx	Las **gráficas** del laboratorio: curvas que muestran cómo crece el tiempo de cada método al aumentar los datos.
Dashboard.jsx	La pestaña **Analítica Urbana**. Gráficas de accidentes por hora, día, mes, tipo y barrio, con filtros, conclusiones automáticas y botón para descargar los datos en CSV (Excel).

Archivo	¿Qué hace? (en palabras simples)
PpiDocs.jsx	La pestaña **Ficha Técnica PPI**. Lista los 10 requisitos del proyecto académico (R1 a R10), si se cumplieron y con qué evidencia.

7. Ejemplo: ¿qué pasa cuando pido una ruta?

Para ver cómo trabajan juntas todas las piezas, sigamos un pedido de principio a fin:

1. Escribes "Tecnológico de Antioquia" como origen y "Universidad de Antioquia" como destino en el mapa (**MapView.jsx**).

2. Mientras escribes, la página le pregunta al servidor qué lugares coinciden (**routing.py** → **data_service.py**) y te muestra sugerencias.

3. Pulsas "calcular". La página envía las coordenadas de A y B al servidor.

4. El servidor usa el **KD-Tree** para encontrar la esquina de la red vial más cercana a A y a B.

5. Con **Dijkstra** busca dos caminos: uno pensando solo en distancia y otro dando más peso a la seguridad.

6. Pide a un servicio público de mapas (OSRM) el trazo exacto por las calles para que la línea se vea bien en el mapa. Si no responde, usa un trazo aproximado.

7. Responde con los dos caminos, su distancia, su nivel de riesgo y cuánto riesgo te ahorras con la ruta segura.

8. La página dibuja ambas rutas con colores distintos para que las compares.

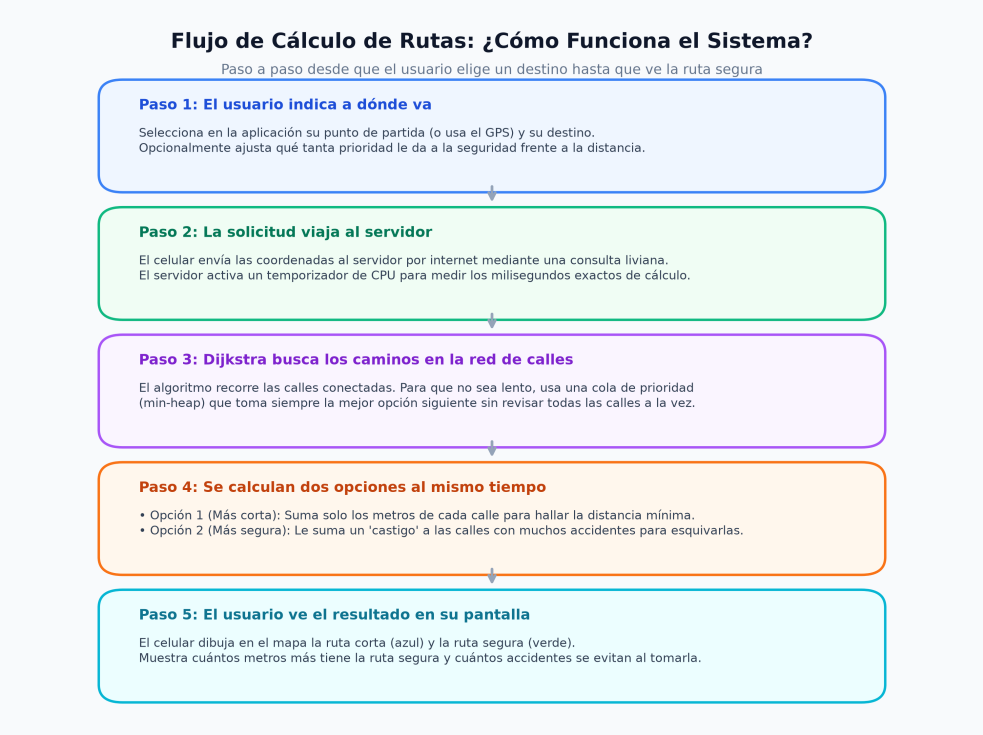


Figura 2. Flujo del cálculo de ruta segura (incluida en la carpeta diagramas/).

8. Las herramientas sueltas (carpeta scripts/)

Son pequeños programas que se ejecutan a mano cuando se necesitan. No forman parte de la app que usa la gente, pero ayudaron a construirla.

8.1 Para preparar los datos

Archivo	¿Qué hace? (en palabras simples)
download_real_data.py	Descarga el archivo de accidentes desde la página oficial de MEData y guarda una nota de verificación.
clean_and_preprocess.py	**Limpia** los datos crudos: quita registros sin ubicación o fuera de Medellín, corrige formatos, y crea el archivo limpio, la base de datos y las muestras de prueba.
build_database.py	Vuelve a crear la base de datos desde el archivo limpio (atajo para usar **db_builder.py**).
build_medellin_graph.py	Construye la **red de calles** de Medellín y le asigna a cada calle su nivel de riesgo según los accidentes que tuvo cerca.
run_benchmarks_fase4.py	Ejecuta todas las mediciones de velocidad grandes (hasta 1 millón de datos) y guarda los resultados.

8.2 Para documentos e informes

Archivo	¿Qué hace? (en palabras simples)
generate_word_report*.py	Generan automáticamente los informes del curso en Word (fases 1, 4 y 5), con tablas y resultados.
update_docx_deliverables.py	Actualiza partes del documento oficial **AburráSegura.docx** (abreviaturas, tablas de objetivos).
generate_software_diagrams.py	Dibuja las imágenes de la carpeta diagramas/.
estandarizar_vault.py y aplicar_titulos.py	Ordenan y ponen títulos a las notas de documentación en la carpeta vault/.

8.3 Para publicar en internet

Archivo	¿Qué hace? (en palabras simples)
deploy_hf_space.py	Sube el servidor a **Hugging Face** (un sitio que lo aloja gratis).

9. El control de calidad (carpeta tests/)

Las pruebas son pequeños programas que **revisan automáticamente** que todo funcione. Es como un inspector que prueba cada plato antes de servirlo. Si alguien cambia algo y rompe una función, las pruebas lo detectan.

Archivo	¿Qué hace? (en palabras simples)
test_sorting.py	Verifica que los cinco métodos de ordenar realmente dejen la lista ordenada.
test_kdtree.py	Verifica que el KD-Tree encuentre el mismo punto más cercano que revisar uno por uno.
test_clustering.py	Verifica que DBSCAN encuentre bien los grupos de accidentes.
test_graph_routing.py / test_routing_risk.py	Verifican que se encuentren caminos correctos y que la ruta segura tenga de verdad menos riesgo.
test_complexity_benchmarks.py / test_benchmarks_fase4.py	Verifican las mediciones del laboratorio.
test_analytics.py	Verifica que las estadísticas y filtros den números correctos.
test_api.py	Verifica que el servidor responda bien a los pedidos.

Se ejecutan todas con un solo comando: **.venv\Scripts\python -m pytest -q**

10. Encender y publicar la aplicación

Archivo	¿Qué hace? (en palabras simples)
start.ps1	El **botón de encendido**. La primera vez instala todo lo necesario; luego enciende el servidor y la página a la vez. Después se abre **http://localhost:5173** en el navegador.
.vscode/tasks.json	Permite encender todo desde Visual Studio Code con **Ctrl + Shift + B**.
requirements.txt / requirements-dev.txt	La lista de herramientas de Python que necesita el servidor (y las de pruebas).
Dockerfile	Una "caja" con instrucciones para montar el servidor en cualquier computador de internet, siempre igual.
deploy-backend.ps1	Publica el servidor en Hugging Face.
deploy-frontend.ps1 y firebase.json	Publican la página web en **Firebase** (servicio de Google).
.gitignore / .gitattributes	Le dicen al sistema de versiones (Git) qué archivos no guardar y cómo tratar los archivos grandes.
README.md	Las instrucciones rápidas del proyecto.
AburráSegura.docx	El documento académico oficial del proyecto.

Nota: el servidor gratuito de Hugging Face se "duerme" tras 48 horas sin visitas. La primera visita después tarda cerca de un minuto; mientras tanto la barra superior muestra "Despertando servidor...".

11. Pequeño diccionario

Archivo	¿Qué hace? (en palabras simples)
Algoritmo	Lista de pasos para resolver un problema. Una receta.
API	La forma en que la página y el servidor se piden cosas. La ventanilla de pedidos.
Backend	La parte invisible que hace los cálculos (la cocina).
Frontend	La parte visible que usa la persona (la sala).
Base de datos	Un archivador digital muy ordenado para buscar información rápido.
CSV	Un archivo de tabla simple que se puede abrir en Excel.
Grafo	Una red de puntos unidos por líneas. Aquí: esquinas unidas por calles.
Coordenadas	La "dirección" exacta de un punto en el planeta (latitud y longitud).

Archivo	¿Qué hace? (en palabras simples)
Benchmark	Una medición de qué tan rápido es algo.
Mapa de calor	Un mapa que pinta más intenso donde hay más accidentes.
Punto negro / hotspot	Un lugar donde se concentran muchos accidentes.
Comuna / barrio	Divisiones territoriales de Medellín.